第 24 章：异步生成器、流式消费与异步资源管理

本章最重要的一句话是：`async for` 解决“怎样等待下一项”，`async with` 解决“谁负责关闭资源”；可靠的流式程序必须同时把这两个问题写清楚。

LLM 回答、HTTP 响应、数据库游标和消息订阅都可能持续返回数据。消费者既可能自然读完，也可能在拿到足够信息后 `break`，还可能处理中途报错或被用户取消。流背后的连接、会话和后台 Task 不能靠垃圾回收碰运气，而要在所属作用域内确定性结束。

可以先把完整生命周期记成一条主线：

```
获取异步会话
    ↓
创建流对象（异步生成器此时还没有执行函数体）
    ↓
逐项等待并消费应用事件
    ↓
自然耗尽 / break / 消费异常 / Task 取消
    ↓
关闭流 → 关闭会话 → 离开请求边界
```

学完本章，你应该能够：

- 展开 `async for` 背后的 `__aiter__()`、`__anext__()` 和 `StopAsyncIteration`；
- 区分 coroutine、async generator 及其正确消费方式；
- 解释 `async for` 为什么不会自动并行，以及直接拉取为什么自带背压；
- 区分网络分块、文本记录、JSON 对象、应用事件和结束信号；
- 使用 `async with`、`asynccontextmanager`、`aclosing` 和 `AsyncExitStack` 管理异步资源；
- 为自然结束、`break`、异常和取消分别指出清理责任人；
- 判断何时直接消费已经足够，何时才需要有界 `Queue` 和独立 producer Task。

本章只使用标准库和内存替身来讲语言协议与所有权。真实 HTTP 流、断线和连接池在第 25 章继续讨论，对外提供 FastAPI 流式接口则留到第 28 章。

24.1 先建立 Java → Python 心智模型

2502x6dg8fpjj0sff40000gp/T/crossnote-f-8cd0b5a6176db83838cef10efe7.html

eclp相关 配运运单26年需求... 运单小秘值班安排 研发日常任务 运单运维 运单-资源大屏 20260618-应急预案 运单26年需求迭代... 商家下单计数

24.1 先建立 Java → Python 心智模型

下面的对照只用于定位概念，不能把两边机械等同：

Java 经验	Python 对应入口	相似点	不能等同的地方
`Iterator<T>` / 增强 `for`	`AsyncIterator[T]` / `async for`	都是逐项请求，耗尽后结束	Java `next()` 同步返回；的下一项要 `await`
`CompletableFuture<T>`	coroutine + `await`	都表示未来的一次结果	coroutine 调用后默认尚未交给 Task 后才并发运行
Java Stream API	async generator	都能形成惰性数据管道	Java Stream 主要是同步集 async generator 允许每次 I/O
Reactive Streams `Publisher<T>`	async iterator	都处理随时间到达的多项数据	Reactive Streams 有订阅! demand 协议；async iter 是消费者主动拉取下一项
`try-with-resources`	`async with`	都把资源生命周期绑定到词法作用域	`AutoCloseable.close()` 同步执行；`__aexit__()` `await`
动态登记多个 `AutoCloseable`	`AsyncExitStack`	都要逆序清理已成功获取的资源	`AsyncExitStack` 能混合[异步退出回调
有界 `BlockingQueue` + executor	`asyncio.Queue` + `TaskGroup`	都能解耦生产者与消费者并限制缓冲	`asyncio.Queue` 服务于同一事件循环中的 不是线程安全队列

尤其不要被单词 `stream` 误导。Java Stream API、网络字节流、LLM 应用事件流和异步生成器是不同层次的概念。本章讨论的是“怎样异步逐项取得数据，以及怎样关闭承载这条数据流的资源”。

24.2 `async for` 到底怎样取得下一项

异步迭代协议

同步 iterator 的 `next()` 必须当场返回值；异步 iterator 的下一项可能还在网络上，因此

:72502x6dg8fpjj0sff40000gp/T/crossnote-f-8cd0b5a6176db83838cef10efe7.html

eclp相关 配运运单26年需求... 运单小秘值班安排 研发日常任务 运单运维 运单-资源大屏 20260618-应急预案 运单26年需求迭代... 商家下单计数

24.2 `async for` 到底怎样取得下一项

异步迭代协议

同步 iterator 的 `next()` 必须当场返回值；异步 iterator 的下一项可能还在网络上，因此 `__anext__()` 返回 awaitable，等待完成后才得到值。

操作	同步迭代	异步迭代
获得 iterator	`iter(source)`	`aiter(source)`
iterable 协议	`__iter__()`	`__aiter__()`
取得下一项	`next(iterator)`	`await anext(iterator)`
下一项方法	`__next__()`	`__anext__()`
正常耗尽	`StopIteration`	`StopAsyncIteration`
循环语法	`for`	`async for`

一个最小异步 iterator 可以这样写：

```
from collections.abc import AsyncIterator
from typing import Self


class AsyncCounter(AsyncIterator[int]):
    def __init__(self, stop: int) -> None:
        self.current = 0
        self.stop = stop

    def __aiter__(self) -> Self:
        return self

    async def __anext__(self) -> int:
        if self.current >= self.stop:
            raise StopAsyncIteration
        value = self.current
        self.current += 1
        return value
```

rf7lz72502x6dg8fpjj0sff40000gp/T/crossnote-f-8cd0b5a6176db83838cef10efe7.html
2.50
3关... eclp相关 配运运单26年需求... 运单小秘值班安排 研发日常任务 运单运维 运单-资源大屏 20260618-应急预案 运单26年需求迭代... 商家下单计数

`__aiter__()` 直接返回异步 iterator，不能返回等待中的 coroutine；`__anext__()` 则必须返回 awaitable，所以通常写成 `async def`。真实实现会在 `__anext__()` 中等待 socket、数据库或消息队列，示例为了聚焦协议直接返回整数。

内置的 `aiter()` 和 `anext()` 可以手动探测这套协议：

```
iterator = aiter(AsyncCounter(2))
first = await anext(iterator)  # 0
second = await anext(iterator)  # 1
end = await anext(iterator, "<已结束>")  # 不抛异常，得到默认值
```

如果 `None` 本身也是合法事件，不要再用 `None` 表示结束。可以让 `StopAsyncIteration` 自然传播，或使用一个不会和业务值冲突的唯一哨兵。

`async for` 的执行顺序

下面的循环：

```
async for event in stream:
    await handle(event)
```

可以用这个核心过程理解：

```
iterator = aiter(stream)
while True:
    try:
        event = await anext(iterator)
    except StopAsyncIteration:
        break
    else:
        await handle(event)
```

关键顺序是：先等待一项，完整执行一次循环体，再请求下一项。`async for` 中的 `async` 表示“取下一项允许等待”，不表示“每一项自动创建一个 Task”。因此它默认保持顺序，也不会自动并行处理多项。

下面这种写法既没有并发上限，也没有 Task 所有者：

'lz72502x6dg8fpjj0sff40000gp/T/crossnote-f-8cd0b5a6176db83838cef10efe7.html

eclp相关 配运运单26年需求... 运单小秘值班安排 研发日常任务 运单运维 运单-资源大屏 20260618-应急预案 运单26年需求迭代... 商家下单计数

下面这种写法既没有并发上限，也没有 Task 所有者：

```
# 反例：快速拉完整条流，并遇留数量无界的后台 Task。
async for event in stream:
    asyncio.create_task(handle(event))
```

若事件必须有序，例如同一 `call_id` 的 `tool-result` 必须晚于 `tool-start`，直接 `await handle(event)` 最清楚。确实需要并发时，应先定义并发上限、结果顺序、失败策略和 Task 所有者，再使用 `TaskGroup` 等结构化并发工具。

24.3 coroutine 与 async generator 不是同一种对象

两者都用 `async def` 定义，但函数体中是否出现 `yield` 会改变函数种类：

```
from collections.abc import AsyncIterator


async def get_answer() -> str:
    return "完整结果"


async def stream_answer() -> AsyncIterator[str]:
    yield "片段一"
    yield "片段二"
```

定义	调用结果	函数体何时开始	正确消费方式
`async def`，没有 `yield`	coroutine 对象	被 `await`、创建为 Task 或交给 runner 时	`answer = await get_answer()`
`async def`，包含 `yield`	async generator 对象	第一次 `anext()` 或进入 `async for` 时	`async for part in stream_answer()`

调用两种函数时都只是创建对象，并不会立刻执行函数体。coroutine 表示“一次最终结果”；async generator 表示“可以异步等待的多项结果”。因此不能直接 await 异步生成器：

```
stream = stream_answer()
answer = await stream  # 类型错误：async generator 不是 awaitable
```

wf7lz72602x6dg8fpjj0sff40000gp/T/crossnote-f-8cd0b5a6176db83838cef10efe7.html
旧关... eclp相关 配运运单26年需求... 运单小秘值班安排 研发日常任务 运单运维 运单-资源大屏 20260618-应急预案 运单26年需求迭代... 商家下单计数

要等待的是“取得下一项”这个动作：

```
stream = stream_answer()
first = await anext(stream)
await stream.aclose()
```

每次 `yield` 都会把一项交给当前消费者，同时保留局部变量和执行位置。下一次 `await anext(stream)` 才从 `yield` 后恢复。函数自然运行到底或执行空 `return` 后，下一次取值表现为 `StopAsyncIteration`。异步生成器不能用 `return value` 额外交付完成值，也不支持 `yield from`。

类型提示表达的是调用方能做什么：

- 参数只要求“能提供异步 iterator”时，使用 `AsyncIterable[Event]`；
- 返回值只承诺“可以逐项读取”时，使用 `AsyncIterator[Event]`；
- 当前代码拥有具体生成器、需要调用 `aclose()` 时，保留 `AsyncGenerator[Event]`。

24.4 网络分块不是应用事件

真实 LLM 流不会天然按 Python 对象边界到达。数据通常经过这些层次：

```
任意大小的网络 bytes chunk
    ↓ 增量解码；一个 UTF-8 字符可能横跨两个 chunk
任意大小的文本片段
    ↓ 按 SSE、JSONL 等 framing 规则累计
完整文本记录
    ↓ JSON 解析和字段校验
JSON 值
    ↓ 按供应商协议转换
TokenEvent / ToolStartEvent / ToolResultEvent / DoneEvent
```

所以不能假设“一次网络读取恰好得到一个 UTF-8 字符串，也恰好得到一个 JSON 对象”：

```
# 反例：chunk 可能切断 UTF-8 字符或 JSON，也可能同时包含多条记录。
event = json.loads(chunk.decode("utf-8"))
```

本章示例从“已经正确解码的文本片段”开始，演示 3 个文本片段最终组成 4 条 JSONL 事件。其中一条 JSON 横跨两个片段，另一个片段又包含两条 JSON。字节级增量解码和 HTTP framing 属于第 25 章。

wf7lz72502x6dg8fpjj0sff40000gp/T/crossnote-f-8cd0b5a6176db83838cef10efe7.html.
相关... eclp相关 配运运单26年需求... 运单小秘值班安排 研发日常任务 运单运维 运单-资源大屏 20260618-应急预案 运单26年需求迭代... 商家下单计数

还要分清四种"结束":

信号	所属层次	它真正表示什么
`DoneEvent(reason="stop")`	LLM 应用协议	模型声明本次生成完成
`StopAsyncIteration`	Python 迭代协议	这个 iterator 没有下一项
`QueueEnd`	进程内 producer/consumer 协议	worker 不必再等待 QueueItem
连接或会话关闭	资源生命周期	底层资源已经释放或归还

收到 `DoneEvent` 不等于连接已经关闭；连接断开时也可能根本收不到 `DoneEvent` 。应用完成、流耗尽、worker 退出和资源关闭必须分别建模。

24.5 `async with` 管理"关闭也需要等待"的资源

同步上下文管理器通过 `__enter__()` / `__exit__()` 进入和退出。异步资源可能要等待建连、鉴权、发送关闭帧、回滚事务或归还异步连接池，因此使用 `__aenter__()` / `__aexit__()`：

```
class AsyncSession:
    async def __aenter__(self):
        await self.connect()
        return self

    async def __aexit__(self, exc_type, exc_value, traceback):
        await self.close()
        return False


async with AsyncSession() as session:
    await session.call()
```

`return False` 表示退出方法不吞掉块内异常。无论块正常结束、抛异常还是收到取消，Python 都会先等待 `__aexit__()` ，然后再离开资源边界。

有一个容易漏掉的边界：如果 `__aenter__()` 自己失败，当前对象的 `__aexit__()` 不会被调用，因为它从未成功进入上下文。因此 `__aenter__()` 必须自行回滚已经完成的部分初始化。

选择 `with` 还是 `async with` ，依据是资源协议，不是外围函数有没有 `async` ：

×
+
ɲ/wf7lz72502x6dg8fpjj0sff40000gp/T/crossnote-f-8cd0b5a6176db83838cef10efe7.html
2.60
求相关... eclp相关 配运运单26年需求... 运单小秘值班安排 研发日常任务 运单运维 运单-资源大屏 20260618-应急预案 运单26年需求迭代... 商家下单计数

选择 `with` 还是 `async with`，依据是资源协议，不是外围函数有没有 `async`：

- 获取和释放都能立即完成，使用 `with`；
- 获取或释放需要等待异步 I/O，使用 `async with`；
- 同步 SDK 内部仍会阻塞时，仅套一层 `async def` 或 `async with` 不会把它变成非阻塞 API。

用 `asynccontextmanager` 表达线性生命周期

如果资源生命周期就是"获取 → 借用 → 关闭"，不必专门写一个类：

```
from collections.abc import AsyncIterator
from contextlib import asynccontextmanager


@asynccontextmanager
async def open_session() -> AsyncIterator[Session]:
    session = Session()
    await session.open()
    try:
        yield session
    finally:
        await session.close()
```

装饰后的函数是异步上下文管理器工厂。进入 `async with` 时执行到 `yield`，把资源借给调用方；退出时从 `yield` 处恢复，并在离开边界前完成 `finally`。

一次上下文必须恰好 `yield` 一次。块内异常会被重新抛到暂停的 `yield` 位置；如果为了记录日志而捕获它，通常必须原样重新抛出：

```
try:
    yield session
except BaseException as error:
    log(error)
    raise
finally:
    await session.close()
```

这里使用 `BaseException` 是为了说明取消也会经过这条路径。捕获 `CancelledError` 或 `BaseException` 后正常返回，会把用户取消伪装成成功。只需要清理而不需要记录时，直接使用 `try/finally` 更清楚。

/wf7lz72602x6dg8fpjj0sff40000gp/T/crossnote-f-8cd0b5a6176db83838cef10efe7.html

相关... eclp相关 配运运单26年需求... 运单小秘值班安排 研发日常任务 运单运维 运单-资源大屏 20260618-应急预案 运单26年需求迭代... 商家下单计数

24.6 为什么 `break` 后仍要关闭流

资源型异步生成器通常把清理放在 `finally`：

```
async def read_events(session):
    try:
        async for event in session.incoming_events():
            yield event
    finally:
        await session.finish_stream()
```

消费者提前退出后，生成器可能仍暂停在 `yield`：

```
stream = read_events(session)
async for event in stream:
    if enough(event):
        break
```

`async for` 只定义怎样取下一项和怎样识别耗尽。它不会为任意 async iterator 猜测并调用 `aclose()`，因为并非所有异步 iterator 都有这个方法。只要 `stream` 仍被引用，它的 `finally` 就可能尚未执行。

具体的 async generator 提供 `aclose()`。等待它会在生成器暂停处注入关闭信号，让 `finally` 有机会完成：

```
try:
    async for event in stream:
        if enough(event):
            break
finally:
    await stream.aclose()
```

`contextlib.aclosing()` 把这条规则直接写进作用域：

```
from contextlib import aclosing


async with aclosing(read_events(session)) as stream:
```

```
async with aclosing(read_events(session)) as stream:
    async for event in stream:
        if enough(event):
            break
# 此处已经 await stream.aclose()
```

它的重要价值不是少写两行代码，而是让迭代和退出代码在同一个 Task、同一份上下文中执行。`asyncio.run()` 关闭事件循环时也会最终处理尚未完成的异步生成器，但那只是入口级安全网，不能替代请求作用域内的资源所有权。

四种退出路径的责任分工

退出路径	生成器可能处于什么状态	谁保证确定性关闭
自然耗尽	运行到结尾，`finally` 已执行	生成器自然结束，外层管理器随后关闭会话
消费者 `break`	仍暂停在最后一次 `yield`	拥有者显式 `aclose()`，或使用 `aclosing()`
循环体抛异常	异常离开消费区间，未必进入生成器	拥有者先关闭流，再让会话退出并传播原异常
Task 被取消	取消可能落在取下一项、循环体或清理中的 await	Task 所有者等待完整清理，并继续传播取消

如果取消恰好发生在生成器的 `__anext__()` 内部，它会直接进入生成器并触发 `finally`。但取消也可能发生在循环体中，所以不能依赖“取消刚好落在生成器内部”来代替明确所有权。

24.7 把流和会话交给同一个所有者

对于“一个请求创建一条流，并独占其会话”的场景，推荐从这个结构开始：

```
from collections.abc import AsyncGenerator, AsyncIterator
from contextlib import aclosing, asynccontextmanager


@asynccontextmanager
async def owned_stream() -> AsyncIterator[AsyncGenerator[Event]]:
    async with open_session() as session:
        stream = read_events(session)
        async with aclosing(stream):
```

× +
7h/wf7lz72502x6dg8fpjj0sff40000gp/T/crossnote-f-8cd0b5a6176db83838cef10efe7.html
求相关... eclp相关 配运运单26年需求... 运单小秘值班安排 研发日常任务 运单运维 运单-资源大屏 20260618-应急预案 运单26年需求迭代... 商家下单计数

24.7 把流和会话交给同一个所有者

对于"一个请求创建一条流，并独占其会话"的场景，推荐从这个结构开始：

```
from collections.abc import AsyncGenerator, AsyncIterator
from contextlib import aclosing, asynccontextmanager


@asynccontextmanager
async def owned_stream() -> AsyncIterator[AsyncGenerator[Event]]:
    async with open_session() as session:
        stream = read_events(session)
        async with aclosing(stream):
            yield stream
```

嵌套顺序表达了关闭顺序：先进入的 session 最后退出，后进入的 stream 最先关闭。消费者只借用 iterator，不承担猜测底层资源结构的责任：

```
async def consume_request() -> None:
    async with owned_stream() as events:
        async for event in events:
            await handle(event)
            if isinstance(event, DoneEvent):
                break
```

无论消费者自然结束、`break`、抛异常还是收到取消，离开 `owned_stream()` 时都会先 `await stream.aclose()`，再等待 session 关闭，最后才把异常或取消交还外层。

可以用一句工程规则记住它：**谁创建并独占资源，谁负责在自己的作用域内关闭；只借用资源的函数不要擅自关闭调用方还要复用的对象。**

若第三方 SDK 已经返回同时管理响应与连接的原生异步上下文管理器，应优先遵循它的官方协议，不要根据方法名猜一套额外关闭逻辑。锁定版本不同，SDK 返回值可能是 AsyncIterator、上下文管理器，或两者的组合。

取消不是"丢掉 Task 引用"

`task.cancel()` 只是请求在某个可取消点抛出 `CancelledError`，不表示资源已经关完。Task 的所有者还要等待它：

rs/7h/wf7lz72502x6dg8fpjj0sff40000gp/T/crossnote-f-8cd0b5a6176db83838cef10efe7.html

的需求相关... eclp相关 配运运单26年需求... 运单小秘值班安排 研发日常任务 运单运维 运单-资源大屏 20260618-应急预案 运单26年需求迭代... 商家下单计数

取消不是“丢掉 Task 引用”

`task.cancel()` 只是请求在某个可取消点抛出 `CancelledError`，不表示资源已经关完。Task 的所有者还要等待它：

```
task.cancel()
try:
    await task
except asyncio.CancelledError:
    pass
```

当 `await task` 返回或抛出取消时，Task 内部的 `finally` 和上下文退出才已经走完。内部代码可以记录取消，但通常不应吞掉它，否则上层会把“用户停止”误判为正常完成。

24.8 直接拉取已经具有自然背压

异步生成器和消费者在同一个 Task 中直接连接时，运行过程是：

```
consumer: await anext(stream)
    ↓
generator: 准备并 yield event-1
    ↓
consumer: 完整处理 event-1
    ↓
consumer: 再次 await anext(stream)
    ↓
generator: 才从上次 yield 后恢复并准备 event-2
```

消费者没有请求下一项时，上游就不会恢复生产，这就是拉取模型的自然背压。顺序处理、无需预取、单项处理足够快时，直接 `async for` 往往已经是最可靠的方案：没有额外 Task，没有缓冲区，也不需要单独设计 producer/consumer 的失败联动。

什么时候才加入有界 Queue

只有确实需要有限预取、独立 producer/consumer 等待，或把来源分发给 worker 时，才值得增加 Queue：

```
produces Task: 唯一拥有并消费客层 stream
```

7h/wf7lz72502x6dg8fpjj0sff40000gp/T/crossnote-f-8cd0b5a6176db83838cef10efe7.html

求相关... eclp相关 配运运单26年需求... 运单小秘值班安排 研发日常任务 运单运维 运单-资源大屏 20260618-应急预案 运单26年需求迭代... 商家下单计数

什么时候才加入有界 Queue

只有确实需要有限预取、独立 producer/consumer 等待，或把来源分发给 worker 时，才值得增加 Queue：

```
producer Task: 唯一拥有并消费底层 stream
        | await queue.put(event)
        ▼
Queue(maxsize=2): 最多缓冲 2 项
        | await queue.get()
        ▼
consumer Task: 处理应用事件

共同的 TaskGroup: 拥有并等待双方
```

这里有四条硬规则：

1. 使用 `maxsize > 0` 。 `asyncio.Queue(maxsize=0)` 表示无界，不是零容量；队列满时 `await put()` 才会把压力传回 producer。
2. producer 和 consumer 必须有共同所有者。只靠正常路径放结束哨兵不够：producer 失败时，consumer 可能永远等不到哨兵；consumer 失败时，producer 可能永远卡在满队列上。
3. 每次成功 `get()` 都要配对一次 `task_done()` ，包括内部结束哨兵。 `await queue.join()` 只检查 Queue 记账归零，不代表业务一定成功，也不会自动停止 worker。
4. `DoneEvent` 和 `QueueEnd` 分开建模。前者属于 LLM 协议，后者只用于本地 worker 生命周期。

最小的消费记账形状是：

```
item = await queue.get()
try:
    if isinstance(item, QueueEnd):
        return
    await handle(item)
finally:
    queue.task_done()
```

本章示例把 producer 与 consumer 放进同一个 `TaskGroup` 。正常路径使用内部哨兵；外部取消时，TaskGroup 取消并等待双方，producer 自己关闭流和会话，管道所有者明确丢弃未消费项目并完成 Queue 记账。这是第 23 章中的有界 Queue 与结构化并发在资源型流上的具体应用。

uf7lz72502x6dg8fpjj0sff40000gp/T/crossnote-f-8cd0b5a6176db83838cef10efe7.html

关... eclp相关 配运运单26年需求... 运单小秘值班安排 研发日常任务 运单运维 运单-资源大屏 20260618-应急预案 运单26年需求迭代... 商家下单计数

24.9 `AsyncExitStack` 管理运行时数量的资源

固定数量的资源直接嵌套最容易读：

```
async with open_http_client() as http, open_mcp_session() as mcp:
    await use(http, mcp)
```

资源数量由配置或运行结果决定时，使用 `AsyncExitStack`：

```
from contextlib import AsyncExitStack


async with AsyncExitStack() as stack:
    sessions = []
    for config in runtime_configs:
        session = await stack.enter_async_context(open_session(config))
        sessions.append(session)
    await use(sessions)
```

每次 `enter_async_context()` 成功，退出方法就被登记入栈。离开时按后进先出顺序关闭；后续某次获取失败时，已经成功进入的资源也会逆序清理。失败资源自己的 `__aenter__()` 尚未成功，所以仍要由它自行回滚部分初始化。

`AsyncExitStack` 还可以组合：

- `stack.enter_context(...)`：同步上下文管理器；
- `await stack.enter_async_context(...)`：异步上下文管理器；
- `stack.callback(...)`：同步清理回调；
- `stack.push_async_callback(...)`：异步清理回调。

异步栈没有可替代清理的同步 `close()`；要通过外层 `async with`，或显式 `await stack.aclose()` 驱动异步退出。

24.10 Java 工程师最容易踩的坑

误判或错误写法	实际表现	正确做法
`await stream_answer()`	async generator 不是 awaitable	`async for` 或 `await anext(...)`

?lz72502x6dg8fpjj0sff40000gp/T/crossnote-f-8cd0b5a6176db83838cef10efe7.html

eclp相关 配运运单26年需求... 运单小秘值班安排 研发日常任务 运单运维 运单-资源大屏 20260618-应急预案 运单26年需求迭代... 商家下单计数

24.10 Java 工程师最容易踩的坑

误判或错误写法	实际表现	正确做法
`await stream_answer()`	async generator 不是 awaitable	`async for` 或 `await anext(...)`
`async for` 会并行处理每项	下一项要等当前循环体结束后才请求	需要并发时显式建立有上限、有所有者的 Task
`break` 会关闭任意 async iterator	循环退出，但生成器可能仍暂停在 `yield`	具体生成器使用 `aclose()` / `aclosing()`
等 `asyncio.run()` 退出再清理	请求级资源可能被占用到进程入口结束	在创建流的局部作用域确定性关闭
`asynccontextmanager` 记录异常后不 `raise`	原异常或取消被误判为已经处理	只记录时原样重新抛出，清理放在 `finally`
一个网络 chunk 直接 `decode` 再 `json.loads`	UTF-8、文本行或 JSON 都可能跨 chunk	增量解码、累计 framing，再解析应用事件
把模型 `done` 当作 worker 哨兵	应用协议与本地生命周期耦合	单独定义 `QueueEnd` 或明确关闭策略
`Queue()` 默认不能缓存	`maxsize=0` 实际是无界	需要容量边界时使用正整数
producer 正常结束时放哨兵就够了	任一方失败都可能让另一方永久等待	用 `TaskGroup` 联动失败、取消并等待收尾
调用 `task_done()` 表示业务成功	它只减少 Queue 的未完成项目计数	业务结果和异常要单独保存、传播
`task.cancel()` 后立刻丢引用	`finally` 可能仍在异步关闭资源	cancel 后继续 await Task
`async with` 能修复同步阻塞 SDK	语法不会改变内部调用方式	使用真正异步 API，或明确把同步工作移出事件循环

做方案选择时，可以从这张表开始：

需求	优先选择
下一项立即可得，不需要异步 I/O	`Iterator[T]`，`for`

/wf7lz72502x6dg8fpjj0sff40000gp/T/crossnote-f-8cd0b5a6176db83838cef10efe7.html
eclp相关 配运运单26年需求... 运单小秘值班安排 研发日常任务 运单运维 运单-资源大屏 20260618-应急预案 运单26年需求迭代... 商家下单计数

做方案选择时，可以从这张表开始：

需求	优先选择
下一项立即可得，不需要异步 I/O	`Iterator[T]` + `for`
取得下一项可能等待	`AsyncIterator[T]` + `async for`
一次异步操作只返回一个最终值	coroutine + `await`
自己实现惰性的异步多项来源	async generator
具体 async generator 可能提前停止	`aclosing()`
获取或关闭资源需要等待	`async with`
线性的异步资源工厂	`@asynccontextmanager`
运行时数量的同步/异步资源	`AsyncExitStack`
顺序消费已经满足需求	直接 `async for`
需要有限预取或独立生产/消费 Task	有界 Queue + `TaskGroup`

24.11 递进练习

建议把练习写在独立文件中，不要修改已经带断言的 `example.py` 。

练习 1：手写协议并展开 `async for`

实现 `AsyncMessageSource` ，依次产生 `"A"` 、 `"B"` ，耗尽时抛 `StopAsyncIteration` 。先手动调用 `aiter()` / `anext()` ，再用 `async for` 消费，并写出两者之间的对应关系。

验收重点： `__aiter__()` 直接返回 iterator；第三次取值分别用异常和默认值两种方式验证； `async for` 得到相同结果。

练习 2：验证提前 `break` 的清理时机

写一个产出三个 token 的 async generator，把清理记录放在 `finally` 。消费第二项后 `break` ，分别观察不调用 `aclose()` 、显式调用 `aclose()` 和使用 `aclosing()` 时的事件顺序。

验收重点：第三项没有生产；离开 `aclosing()` 前流仍在借用区间内，离开后 `finally` 已执行且只

wf7lz72502x6dg8fpjj0sff40000gp/T/crossnote-f-8cd0b5a6176db83838cef10efe7.html
旧关... eclp相关 配运运单26年需求... 运单小秘值班安排 研发日常任务 运单运维 运单-资源大屏 20260618-应急预案 运单26年需求迭代... 商家下单计数

练习 2：验证提前 `break` 的清理时机

写一个产出三个 token 的 async generator，把清理记录放在 `finally`。消费第二项后 `break`，分别观察不调用 `aclose()`、显式调用 `aclose()` 和使用 `aclosing()` 时的事件顺序。

验收重点：第三项没有生产；离开 `aclosing()` 前流仍在借用区间内，离开后 `finally` 已执行且只执行一次。

练习 3：覆盖四种退出路径

用 `@asynccontextmanager` 同时拥有一个 FakeSession 和异步生成器，分别运行自然结束、消费者 `break`、消费者抛异常、外部取消。

验收重点：四条路径都只关闭流一次、会话一次；流关闭早于会话；异常和 `CancelledError` 在清理完成后仍能到达外层。

练习 4：决定是否需要 Queue

先用直接 `async for` 证明慢消费者处理第一项时生成器不会生产第二项。再改成 `Queue(maxsize=2)` 和两个 Task，让 producer 在第三次 `put()` 上确定等待。

验收重点：producer 与 consumer 由同一个 `TaskGroup` 拥有；正常、消费者失败和外部取消都不遗留 Task；每次 `get()` 都有一次 `task_done()`；应用 `DoneEvent` 与内部结束哨兵不是同一对象。

24.12 本章小结与运行方式

只记住下面六条，本章的主线就不会丢：

1. coroutine 是一次异步结果，async generator 是可异步拉取的多项结果。
2. `async for` 反复执行 `await anext(...)`，默认顺序消费，不会自动并行。
3. 网络分块、应用事件、iterator 耗尽和资源关闭是不同层次的事实。
4. `break` 只退出循环；拥有具体 async generator 的代码要确定性 `aclose()`。
5. `async with` 管资源边界，`asynccontextmanager` 管线性生命周期，`AsyncExitStack` 管动态数量。
6. 直接拉取已有自然背压；增加 Queue 时，也必须增加容量、结束协议和 Task 所有权。

前置章节与环境

本章承接以下内容：

rs/7h/wf7lz72502x6dg8fpjj0sff40000gp/T/crossnote-f-8cd0b5a6176db83838cef10efe7.html

的需求相关... eclp相关 配运运单26年需求... 运单小秘值班安排 研发日常任务 运单运维 运单-资源大屏 20260618-应急预案 运单26年需求迭代... 商家下单计数

前置章节与环境

本章承接以下内容：

- 第 17 章：iterator、生成器与惰性生产；
- 第 18 章：`with` 、 `contextmanager` 与资源所有权；
- 第 19 章：coroutine、 `await` 与事件循环；
- 第 20～22 章：Task 生命周期、取消与 `TaskGroup` ；
- 第 23 章：Queue、背压与结构化并发。

根 `pyproject.toml` 要求 Python `>=3.14` ，Ruff 和 Pyright 也以 Python 3.14 为目标。本章只使用标准库，没有新增依赖。当前核对环境为：

工具	版本	用途
CPython	3.14.7	运行完整示例与断言
Ruff	0.16.5	检查并格式化 Python 代码
Pyright	1.1.411	basic 模式静态类型检查

`aiter()` 、 `anext()` 和 `aclosing()` 从 Python 3.10 可用， `TaskGroup` 从 3.11 可用；本章示例按项目的 3.14 约束编写，不承诺兼容更旧版本。

运行完整示例

本章文件只有两个：

文件	职责
`README.md`	概念、心智模型、所有权规则与练习
`example.py`	九组离线实验、中文生命周期轨迹与运行时断言

在仓库根目录运行：

```
.venv/bin/python -B java_to_python/chapters/24_async_streams_resources/example.py
```

`-B` 只是不生成 `.pyc` ，不改变异步语义。 `java_to_python/run_all.py` 目前只运行前 10 章，本章没有接入。

×
+
←s/7h/wf7lz72502x6dg8fpjj0sff40000gp/T/crossnote-f-8cd0b5a6176db83838cef10efe7.html
的需求相关... eclp相关 配运运单26年需求... 运单小秘值班安排 研发日常任务 运单运维 运单-资源大屏 20260618-应急预案 运单26年需求迭代... 商家下单计数

运行完整示例

本章文件只有两个：

文件	职责
`README.md`	概念、心智模型、所有权规则与练习
`example.py`	九组离线实验、中文生命周期轨迹与运行时断言

在仓库根目录运行：

```
.venv/bin/python -B java_to_python/chapters/24_async_streams_resources/example.py
```

`-B` 只是不生成 `.pyc`，不改变异步语义。`java_to_python/run_all.py` 目前只运行前 10 章，本章没有接入。

完整入口依次验证：

1. 手动 `aiter/anext`，以及 coroutine 与 async generator 的对象差异；
2. 3 个文本片段组成 4 个 JSONL 应用事件；
3. 直接 `async for` 的生产/消费交替与自然关闭；
4. `break`、显式 `aclose()` 和消费者异常的清理顺序；
5. 外部取消先关闭流和会话，再把 `CancelledError` 交还调用方；
6. `AsyncExitStack` 在后续获取失败时逆序清理已进入资源；
7. 慢消费者对直接拉取形成自然背压；
8. `Queue(maxsize=2)` 满时 producer 等待；
9. Queue 管道取消后 producer、consumer、stream、session 和 Queue 记账全部回收。

示例结束时应看到：

```
入口结束前未回收的其他 Task 数: 0
所有第 24 章示例与运行时断言均已完成。
```

项目中已配置的检查命令：

```
.venv/bin/ruff check java_to_python/chapters/24_async_streams_resources
.venv/bin/ruff format —check java_to_python/chapters/24_async_streams_resources
.venv/bin/pyright java_to_python/chapters/24_async_streams_resources/example.py
```

× +
ers/7h/wf7lz72502x6dg8fpjj0sff40000gp/T/crossnote-f-8cd0b5a6176db83838cef10efe7.html
的需求相关... eclp相关 配运运单26年需求... 运单小秘值班安排 研发日常任务 运单运维 运单-资源大屏 20260618-应急预案 运单26年需求迭代... 商家下单计数

`-B` 只是不生成 `.pyc`，不改变异步语义。`java_to_python/run_all.py` 目前只运行前 10 章，本章没有接入。

完整入口依次验证：

1. 手动 `aiter/anext`，以及 coroutine 与 async generator 的对象差异；
2. 3 个文本片段组成 4 个 JSONL 应用事件；
3. 直接 `async for` 的生产/消费交替与自然关闭；
4. `break`、显式 `aclose()` 和消费者异常的清理顺序；
5. 外部取消先关闭流和会话，再把 `CancelledError` 交还调用方；
6. `AsyncExitStack` 在后续获取失败时逆序清理已进入资源；
7. 慢消费者对直接拉取形成自然背压；
8. `Queue(maxsize=2)` 满时 producer 等待；
9. Queue 管道取消后 producer、consumer、stream、session 和 Queue 记账全部回收。

示例结束时应看到：

```
入口结束前未回收的其他 Task 数：0
所有第 24 章示例与运行时断言均已完成。
```

项目中已配置的检查命令：

```
.venv/bin/ruff check java_to_python/chapters/24_async_streams_resources
.venv/bin/ruff format --check java_to_python/chapters/24_async_streams_resources
.venv/bin/pyright java_to_python/chapters/24_async_streams_resources/example.py
```

这些离线实验验证 Python 协议、清理顺序、取消传播、Queue 容量和 Task 所有权，不验证真实网络中的 DNS、TCP、TLS、HTTP framing、连接复用、SDK 事件 schema 或远端是否响应取消。接入真实客户端后，仍应根据锁定版本的官方文档和集成测试确认流边界与关闭行为。

继续核对语言细节时，可直接查阅：

- Python 3.14 异步迭代器数据模型；
- Python 3.14 `async for` 与 `async with`；
- `asynccontextmanager`、`aclosing` 与 `AsyncExitStack`；
- `asyncio.Queue`；
- `asyncio.TaskGroup`。

```python
"""第 24 章：异步生成器、流式消费与异步资源管理。

全部实验只使用标准库、内存事件和可控闸门，不访问真实 LLM 或网络。
事件轨迹与断言关注协议、所有权和清理顺序，不依赖精确耗时或偶然调度顺序。
"""

# 建议按入口中的 1～9 段顺序阅读：先展开异步迭代协议，再建立事件流和资源所有权，
# 最后比较“直接拉取”与“Queue 解耦”两种生产/消费结构。数据主线都是：
# FAKE_EVENTS -> 异步生成器 -> 消费者；events 列表则在旁边保存生命周期轨迹供断言检查。
# 导入按用途理解：asyncio 提供事件循环、Task、Event、Queue 与 TaskGroup，inspect 用来辨认
# 调用后得到的对象，json 只把完整 JSON 文本解析成 Python 对象，不负责处理传输分片。
# collections.abc 提供迭代协议类型；contextlib 提供异步资源管理工具；dataclasses 与 Enum
# 用来声明事件和状态。TracebackType、Self 则让上下文协议方法的类型提示更准确。
import asyncio
import inspect
import json
from collections.abc import AsyncGenerator, AsyncIterator, Iterable, Iterator
from contextlib import AsyncExitStack, aclosing, asynccontextmanager
from dataclasses import dataclass, field
from enum import Enum, auto
from types import TracebackType
from typing import Self


# 这四个 dataclass 是应用层事件，不是网络 chunk 或 JSON 字典。
# frozen=True 禁止字段重新赋值，slots=True 不再为每个实例创建可任意扩展的 __dict__；
# 两者都不负责运行时类型校验，传入值是否符合注解仍由调用方保证。
@dataclass(frozen=True, slots=True)
class TokenEvent:
    # 一次可展示的文本增量，例如“正在”；它不承诺对应一个网络分块。
    text: str


@dataclass(frozen=True, slots=True)
class ToolStartEvent:
    # name 标识工具，call_id 把开始事件与稍后的结果事件关联起来。
    name: str
```

```python
@dataclass(frozen=True, slots=True)
class ToolStartEvent:
    # name 标识工具，call_id 把开始事件与稍后的结果事件关联起来。
    name: str
    call_id: str


@dataclass(frozen=True, slots=True)
class ToolResultEvent:
    name: str
    call_id: str
    # 为简化实验，工具输出直接保存为字符串；真实协议可能还要建模结构化结果和错误。
    output: str


@dataclass(frozen=True, slots=True)
class DoneEvent:
    # done 是应用协议中的完成通知，不等于 iterator 已耗尽，也不等于连接已经关闭。
    reason: str


# Python 3.12+ 的 type 语句给联合类型命名。LLMEvent 表示四类对象之一；
# 类型检查器可据此检查分支是否覆盖事件，但运行时不会自动验证赋值。
type LLMEvent = TokenEvent | ToolStartEvent | ToolResultEvent | DoneEvent


# tuple[LLMEvent, ...] 表示元素都是 LLMEvent、长度不限的元组。元组本身不可增删，
# 配合不可变事件对象，让九组实验安全复用同一份固定输入。
FAKE_EVENTS: tuple[LLMEvent, ...] = (
    TokenEvent("正在"),
    TokenEvent("查询"),
    ToolStartEvent("weather", "call-1"),
    ToolResultEvent("weather", "call-1", "晴, 25°C"),
    TokenEvent("。"),
    DoneEvent("stop"),
)
```

```python

# 普通 def 足够完成纯同步输出；async 函数也可以调用它，不必把所有辅助函数都写成 async def。
def print_section(title: str) -> None:
    # f-string 会先求值花括号内的 title，再拼进字符串；开头的 \n 让章节之间空一行。
    print(f"\n=== {title} ===")


def record(events: list[str], message: str) -> None:
    """事件既打印又保存，便于人眼观察并用断言验证因果关系。"""
    # append 原地修改调用方传入的同一个 list；没有复制，因此稍后的断言能看到完整轨迹。
    events.append(message)
    print(f"  事件 | {message}")


def event_label(event: LLMEvent) -> str:
    # match/case 做结构模式匹配：类模式既检查实例类型，也按 dataclass 字段顺序解构属性。
    # 例如 TokenEvent("正在") 会匹配第一支，并把 text 绑定为 "正在"。
    match event:
        case TokenEvent(text):
            # !r 使用 repr 格式，文本边界和转义字符会比普通插值更容易观察。
            return f"token({text!r})"
        case ToolStartEvent(name, call_id):
            return f"tool-start({name}, {call_id})"
        case ToolResultEvent(name, call_id, output):
            return f"tool-result({name}, {call_id}, {output!r})"
        case DoneEvent(reason):
            return f"done({reason})"


# 继承 AsyncIterator[int] 主要表达类型契约：每次异步取值最终得到 int。
# 真正让 async for 工作的是下面两个特殊方法，而不是继承动作本身。
class ManualAsyncRange(AsyncIterator[int]):
    """最小异步迭代器：每次 __anext__ 都返回一个需要 await 的对象。"""

    def __init__(self, stop: int, events: list[str]) -> None:
        # iterator 必须保存当前位置；_ 前缀只是“不公开使用”的命名约定，并非访问权限。
        self._next_value = 0
```

```python

# 继承 AsyncIterator[int] 主要表达类型契约：每次异步取值最终得到 int。
# 真正让 async for 工作的是下面两个特殊方法，而不是继承动作本身。
class ManualAsyncRange(AsyncIterator[int]):
    """最小异步迭代器：每次 __anext__ 都返回一个需要 await 的对象。"""

    def __init__(self, stop: int, events: list[str]) -> None:
        # iterator 必须保存当前位置；_ 前缀只是“不公开使用”的命名约定，并非访问权限。
        self._next_value = 0
        self._stop: int = stop
        # 保存外部列表的引用，让协议方法与演示函数把记录写到同一条轨迹中。
        self._events: list[str] = events

    def __aiter__(self) -> Self:
        # __aiter__ 同步返回异步 iterator 本身，不能返回等待中的 coroutine。
        record(self._events, "协议：__aiter__ 返回异步迭代器自己")
        return self

    async def __anext__(self) -> int:
        # async def 调用后先产生可等待对象；async for 会 await 它，得到一项或耗尽信号。
        record(self._events, f"协议：请求下一项，当前位置={self._next_value}")
        # 真实实现可能在这里等待 socket 或数据库；sleep(0) 只提供协作检查点。
        await asyncio.sleep(0)
        if self._next_value >= self._stop:
            # StopAsyncIteration 是协议的正常结束信号，async for 会捕获它并结束循环。
            record(self._events, "协议：抛出 StopAsyncIteration")
            raise StopAsyncIteration

        # 先保存本次返回值再推进游标，确保依次得到 0、1，而非提前加一后的值。
        value: int = self._next_value
        self._next_value += 1
        record(self._events, f"协议：返回 {value}")
        return value


# 这个 async def 不含 yield，所以它是 coroutine 函数：一次 await 最终取得一个 str。
async def coroutine_answer(events: list[str]) -> str:
```

```python

# 这个 async def 不含 yield，所以它是 coroutine 函数：一次 await 最终取得一个 str。
async def coroutine_answer(events: list[str]) -> str:
    record(events, "coroutine: 函数体运行")
    return "最终值"


# 函数体含 yield，因此它是异步生成器函数。调用时只创建 AsyncGenerator，
# 第一次 await anext(...) 或进入 async for 时才真正执行函数体。
async def tiny_async_generator(events: list[str]) -> AsyncGenerator[str]:
    try:
        record(events, "async generator: 函数体运行")
        # yield 交出一项并暂停在这里，同时保留局部状态，等待消费者请求下一项或关闭。
        yield "流中的一项"
    finally:
        # 自然耗尽、显式 aclose 或进入生成器的异常/取消都可能让这个清理块执行。
        record(events, "async generator: finally 清理")


async def demonstrate_protocol_and_object_kinds() -> None:
    print_section("1. AsyncIterator 协议与 coroutine / async generator")
    events: list[str] = []
    source = ManualAsyncRange(2, events)
    # aiter(source) 调用 source.__aiter__()；本类返回自身，所以两个名字指向同一对象。
    iterator: ManualAsyncRange = aiter(source)
    # is 比较对象身份，不是内容是否相等。
    assert iterator is source

    # anext(iterator) 返回 awaitable，await 后才拿到 int。第三次传入默认值后，
    # 内部 StopAsyncIteration 会被 anext 转换为默认字符串，不向外抛出。
    first: int = await anext(iterator)
    second: int = await anext(iterator)
    exhausted: int | str = await anext(iterator, "<已耗尽>")
    print("  手动 aiter/anext:", first, second, exhausted)

    # async for 可理解为：先 aiter 一次，再反复 await anext；只有取值步骤能够暂停，
```

```python
async def demonstrate_protocol_and_object_kinds() -> None:
    # async for 可理解为：先 aiter 一次，再反复 await anext；只有取值步骤能够暂停，
    # 循环体仍是一项处理完以后才会请求下一项，并不会自动为各项创建 Task。
    async_for_values: list[int] = []
    async for value in ManualAsyncRange(2, events):
        async_for_values.append(value)
        record(events, f"async for 循环体: 消费 {value}")
    print("  async for 结果:", async_for_values)
    assert async_for_values == [0, 1]

    # 下面两次调用都只创建对象，尚未执行函数体，因此 kind_events 仍为空。
    kind_events: list[str] = []
    coroutine_object: CoroutineType[Any, Any, str] = coroutine_answer(kind_events)
    generator_object: AsyncGenerator[str, None] = tiny_async_generator(kind_events)
    print("  调用两个函数后，函数体事件数:", len(kind_events))
    print("  coroutine 对象可 await:", inspect.isawaitable(coroutine_object))
    print("  async generator 对象可 await:", inspect.isawaitable(generator_object))
    print("  async generator 对象是 AsyncIterator:", isinstance(generator_object, AsyncIterator))
    # inspect 同时从“函数种类”和“调用结果能否 await”两个角度验证区别。
    assert inspect.iscoroutinefunction(coroutine_answer)
    assert inspect.isasyncgenfunction(tiny_async_generator)
    assert inspect.isawaitable(coroutine_object)
    assert not inspect.isawaitable(generator_object)

    # coroutine 本身可 await 一次取得最终值；异步生成器不能直接 await，
    # 要 await 它的 anext/aclose 等协议操作。aclose 会使暂停处进入 finally。
    answer: str = await coroutine_object
    item: str = await anext(generator_object)
    await generator_object.aclose()
    print("  await coroutine 得到:", answer)
    print("  anext(async generator) 得到:", item)
    assert kind_events == [
        "coroutine: 函数体运行",
        "async generator: 函数体运行",
        "async generator: finally 清理",
    ]

```

```python
def parse_event(raw: object) -> LLMEvent:
    """在 JSON 边界做最小校验，再建立明确的应用事件类型。"""
    # 参数写成 object，表示边界输入尚不可信；先缩小为 dict，才能安全调用 get。
    if not isinstance(raw, dict):
        raise TypeError("事件必须是 JSON object")

    # dict.get 在键不存在时返回 None，便于把缺字段与类型错误统一判为协议不合格。
    event_type: Any | None = raw.get("type")
    if event_type == "token" and isinstance(raw.get("text"), str):
        return TokenEvent(raw["text"])
    if event_type == "tool-start":
        # 先把字段取到局部变量，再用 isinstance 做运行时检查；类型提示本身不会校验 JSON。
        name: Any | None = raw.get("name")
        call_id: Any | None = raw.get("call_id")
        if isinstance(name, str) and isinstance(call_id, str):
            return ToolStartEvent(name, call_id)
    if event_type == "tool-result":
        name: Any | None = raw.get("name")
        call_id: Any | None = raw.get("call_id")
        output: Any | None = raw.get("output")
        if isinstance(name, str) and isinstance(call_id, str) and isinstance(output, str):
            return ToolResultEvent(name, call_id, output)
    if event_type == "done" and isinstance(raw.get("reason"), str):
        return DoneEvent(raw["reason"])
    # !r 保留原始对象的可辨认表示，便于定位具体是哪份边界数据不符合约定。
    raise ValueError(f"事件字段不符合约定: {raw!r}")


def decode_json_lines(text_fragments: Iterable[str], events: list[str]) -> Iterator[LLMEvent]:
    """把已解码的任意文本分片重组为 JSONL；分片本身不是应用事件。"""
    # buffer 保存尚未组成完整行的尾部。该函数从 str 开始，不涉及 bytes 或 UTF-8 增量解码。
    buffer: Literal[''] = ""
    # enumerate(..., start=1) 同时给出从 1 开始的展示编号和当前文本分片。
    for fragment_number, fragment in enumerate(text_fragments, start=1):
        record(events, f"传输替身: 收到文本分片 {fragment_number}，长度={len(fragment)}")
        # 一个 JSON 行可以横跨分片，所以必须先累计，不能对每个 fragment 直接 json.loads。
```

```python
def decode_json_lines(text_fragments: Iterable[str], events: list[str]) -> Iterator[LLMEvent]:
    """把已解码的任意文本分片重组为 JSONL；分片本身不是应用事件。"""
    # buffer 保存尚未组成完整行的尾部。该函数从 str 开始，不涉及 bytes 或 UTF-8 增量解码。
    buffer: Literal[''] = ""
    # enumerate(..., start=1) 同时给出从 1 开始的展示编号和当前文本分片。
    for fragment_number, fragment in enumerate(text_fragments, start=1):
        record(events, f"传输替身：收到文本分片 {fragment_number}，长度={len(fragment)}")
        # 一个 JSON 行可以横跨分片，所以必须先累计，不能对每个 fragment 直接 json.loads。
        buffer += fragment

        # 同一分片也可能含多行；只要 buffer 还有换行符，就继续拆出最前面的完整记录。
        while "\n" in buffer:
            # maxsplit=1 只切第一处：line 交给解析器，其余完整行和残片继续留在 buffer。
            line: str, buffer: str = buffer.split("\n", maxsplit=1)
            if not line:
                # 空行不代表应用事件，本例选择直接跳过。
                continue
            # json.loads 返回的运行时类型未知，所以先标作 object，再交给 parse_event 校验。
            raw: object = json.loads(line)
            event: TokenEvent | ToolStartEvent | ToolResu... = parse_event(raw)
            record(events, f"解析器：组成完整应用事件 {event_label(event)}")
            # decode_json_lines 是同步生成器：list(...) 或 for 每请求一次，才交出一个完整事件。
            yield event

    # 输入用尽却仍有残片，说明传输结束在一条 JSON 记录中间；不能静默丢失。
    if buffer:
        raise ValueError("流结束时仍有不完整 JSON 文本")


def demonstrate_transport_boundaries() -> None:
    print_section("2. 文本分片、JSON 行与应用事件不是一一对应")
    events: list[str] = []
    # 这三个 tuple 元素故意不按事件边界切分：第一个在 tool-start JSON 中间结束，
    # 第三个则一次带来 tool-result 与 done 两行。相邻字符串字面量会在解析时自动拼接。
    fragments = (
        '{"type":"token","text":"你"}\n{"type":"tool',
```

```
261
262 def demonstrate_transport_boundaries() -> None:
263     print_section("2. 文本分片、JSON 行与应用事件不是一一对应")
264     events: list[str] = []
265     # 这三个 tuple 元素故意不按事件边界切分：第一个在 tool-start JSON 中间结束，
266     # 第三个则一次带来 tool-result 与 done 两行。相邻字符串字面量会在解析时自动拼接。
267     fragments = (
268         '{"type":"token","text":"你"}\n{"type":"tool',
269         '-start","name":"weather","call_id":"c1"}\n',
270         (
271             '{"type":"tool-result","name":"weather","call_id":"c1","output":"晴"}\n'
272             '{"type":"done","reason":"stop"}\n'
273         ),
274     )
275     # decode_json_lines 惰性地产生事件；list 会把它消费到结束并收集全部结果。
276     decoded: list[LLMEvent] = list(decode_json_lines(fragments, events))
277     print("  3 个文本分片最终组成的应用事件数:", len(decoded))
278     print("  应用事件:", [event_label(event) for event in decoded])
279     assert len(fragments) == 3
280     assert len(decoded) == 4
281     assert isinstance(decoded[0], TokenEvent)
282     assert isinstance(decoded[-1], DoneEvent)
283
284
285 # Enum 把会话状态限制为三个有名字的成员；auto() 自动生成此处无需关心的成员值。
286 # 后面用 is 比较枚举成员身份，避免拿裸字符串在各处拼写状态。
287 class SessionState(Enum):
288     NEW = auto()
289     OPEN = auto()
290     CLOSED = auto()
291
292
293 class SessionOpenError(RuntimeError):
294     """模拟异步握手失败。"""
295
296
```

```
296
297 class ConsumerError(RuntimeError):
298     """模拟消费者处理某个事件时失败。"""
299
300
301 # 这个对象集中保存资源是否关闭、关闭次数和产出数量，供不同 Task 共享并供实验断言。
302 @dataclass(slots=True)
303 class StreamLifecycle:
304     # default_factory 传入“创建函数”而不是一个 Event 实例；每次 StreamLifecycle() 都会
305     # 各自调用 asyncio.Event()，因此不同实验不会意外共享同一个闸门。
306     stream_closed: asyncio.Event = field(default_factory=asyncio.Event)
307     session_closed: asyncio.Event = field(default_factory=asyncio.Event)
308     stream_close_count: int = 0
309     session_close_count: int = 0
310     produced_count: int = 0
311
312
313 class FakeAsyncSession:
314     """只模拟可等待的会话获取与关闭，不代表任何真实客户端。"""
315
316     def __init__(
317         self,
318         name: str,
319         events: list[str],
320         lifecycle: StreamLifecycle,
321         *,
322         fail_on_enter: bool = False,
323     ) -> None:
324         # 参数列表中的 * 使 fail_on_enter 成为仅限关键字参数，调用处必须写出参数名。
325         self.name: str = name
326         self.events: list[str] = events
327         self.lifecycle: StreamLifecycle = lifecycle
328         self.fail_on_enter: bool = fail_on_enter
329         self.state: Literal[SessionState.NEW] = SessionState.NEW
330         self.exit_calls = 0
331
332     async def __aenter__(self) -> Self:
```

```python
class FakeAsyncSession:

    async def __aenter__(self) -> Self:
        # async with 会 await __aenter__，并把成功返回的 self 绑定给 as 后面的变量。
        record(self.events, f"{self.name}: __aenter__ 开始获取会话")
        # 真实客户端可能等待连接、TLS 或鉴权；本地替身只让出一次执行权。
        await asyncio.sleep(0)
        if self.fail_on_enter:
            # __aenter__ 失败时，本对象的 __aexit__ 不会被调用，所以进入方法必须自己回滚
            # 已取得的部分状态。本例同时设置 Event 和计数，表示回滚已完成。
            self.state: Literal[SessionState.CLOSED] = SessionState.CLOSED
            self.lifecycle.session_close_count += 1
            self.lifecycle.session_closed.set()
            record(self.events, f"{self.name}: 获取失败并回滚部分状态")
            raise SessionOpenError(f"{self.name}: 握手失败")

        self.state: Literal[SessionState.OPEN] = SessionState.OPEN
        record(self.events, f"{self.name}: 会话已获取")
        return self

    async def __aexit__(
        self,
        exc_type: type[BaseException] | None,
        exc_value: BaseException | None,
        traceback: TracebackType | None,
    ) -> bool:
        # 块正常结束时三个异常参数都是 None；块内失败或取消时会收到对应异常信息。
        # 本例只观察 exc_type，exc_value/traceback 留在签名中以完整实现异步上下文协议。
        self.exit_calls += 1
        exception_name: str = "None" if exc_type is None else exc_type.__name__
        record(self.events, f"{self.name}: __aexit__ 收到 {exception_name}，开始关闭")
        # 关闭网络会话也可能需要发送协议帧或等待连接池；因此退出方法可 await。
        await asyncio.sleep(0)
        self.state: Literal[SessionState.CLOSED] = SessionState.CLOSED
        self.lifecycle.session_close_count += 1
        self.lifecycle.session_closed.set()
        record(self.events, f"{self.name}: 会话已关闭")
```

```
313  class FakeAsyncSession:
350      async def __aexit__(
364          self.lifecycle.session_close_count += 1
365          self.lifecycle.session_closed.set()
366          record(self.events, f"{self.name}: 会话已关闭")
367          # 返回 False 表示“不抑制异常”，原异常会在清理完成后继续向调用方传播。
368          return False
369
370      def assert_open(self) -> None:
371          # is 用于比较唯一的枚举成员；流每次产出前检查，防止在会话关闭后继续使用它。
372          if self.state is not SessionState.OPEN:
373              raise RuntimeError(f"{self.name}: 流只能使用已打开会话")
374
375
376  # StreamPause 是可控取消实验的双向握手：生成器用 waiting 通知“已经停在目标位置”，
377  # 测试方可在此时取消任务，或通过 release 明确放行，而不依赖碰运气的 sleep 时长。
378  @dataclass(slots=True)
379  class StreamPause:
380      before_index: int
381      waiting: asyncio.Event = field(default_factory=asyncio.Event)
382      release: asyncio.Event = field(default_factory=asyncio.Event)
383
384
385  async def fake_llm_stream(
386      session: FakeAsyncSession,
387      events: list[str],
388      lifecycle: StreamLifecycle,
389      *,
390      source_events: tuple[LLMEvent, ...] = FAKE_EVENTS,
391      pause: StreamPause | None = None,
392  ) -> AsyncGenerator[LLMEvent]:
393      """逐项产生 LLM 风格应用事件，并在任何退出路径记录流关闭。"""
394      # 虽然这是函数体第一句，但调用 fake_llm_stream(...) 时不会立刻执行；第一次请求下一项时
395      # 才检查会话并记录“流：开始”。这正是异步生成器的惰性执行特征。
396      session.assert_open()
397      record(events, "流：开始")
398      try:
```

```
385 async def fake_llm_stream(
398     try:
399         # enumerate 同时提供从 0 开始的位置和事件；位置用来精确选择暂停点。
400         for index, event in enumerate(source_events):
401             if pause is not None and index == pause.before_index:
402                 record(events, f"流: 产出第 {index + 1} 项前等待外部条件")
403                 # 先 set 告知外部“已经到达”，再等待 release。Event 被 set 后的 wait 会立即通过。
404                 pause.waiting.set()
405                 await pause.release.wait()
406 
407             # 只模拟“准备下一事件需要异步等待”，不模拟真实 token 到达间隔。
408             await asyncio.sleep(0)
409             session.assert_open()
410             lifecycle.produced_count += 1
411             record(events, f"流: 产出 {event_label(event)}")
412             # yield 把当前 event 交回 anext/async for，并暂停整个生成器；消费者处理期间，
413             # 下面的“从 yield 后恢复”和下一轮生产都不会发生，这就形成直接拉取的自然背压。
414             yield event
415             record(events, "流: 消费者请求下一项，生成器从 yield 后恢复")
416 
417         record(events, "流: 自然结束")
418     finally:
419         # finally 统一覆盖自然耗尽、aclose、生成器内异常，以及发生在当前 __anext__ 中的取消。
420         lifecycle.stream_close_count += 1
421         lifecycle.stream_closed.set()
422         record(events, "流: 已关闭")
423 
424 
425 # 装饰器把“恰好 yield 一次”的异步生成器函数转换为异步上下文管理器工厂。
426 # yield 之前负责获取资源，yield 的值借给 async-with 块，yield 之后负责退出和清理。
427 @asynccontextmanager
428 async def owned_llm_stream(
429     name: str,
430     events: list[str],
431     lifecycle: StreamLifecycle,
432     *,
```

```python
# 装饰器把“恰好 yield 一次”的异步生成器函数转换为异步上下文管理器工厂。
# yield 之前负责获取资源，yield 的值借给 async-with 块，yield 之后负责退出和清理。
@asynccontextmanager
async def owned_llm_stream(
    name: str,
    events: list[str],
    lifecycle: StreamLifecycle,
    *,
    source_events: tuple[LLMEvent, ...] = FAKE_EVENTS,
    pause: StreamPause | None = None,
) -> AsyncIterator[AsyncGenerator[LLMEvent]]:
    """拥有会话和流：离开一次 async-with 就确定性关闭二者。"""
    # 外层 session 先进入、后退出；内层 aclosing 因为后进入，所以会先关闭 stream。
    async with FakeAsyncSession(name, events, lifecycle) as session:
        # 此时只创建异步生成器对象，函数体尚未执行；所有者保留具体对象以便调用 aclose。
        stream: AsyncGenerator[LLMEvent, None] = fake_llm_stream(
            session,
            events,
            lifecycle,
            source_events=source_events,
            pause=pause,
        )
        # aclosing 的 __aexit__ 会 await stream.aclose()。因此消费者 break、抛异常或被取消时，
        # 流都在 session 仍为 OPEN 的范围内确定性关闭，而不是等垃圾回收或 runner 兜底。
        async with aclosing(stream):
            record(events, "所有者：把流借给消费者")
            try:
                # yield 把 stream 借给调用方的 as 变量，并暂停管理器在这里等待块结束。
                yield stream
            except BaseException as error:
                # asynccontextmanager 会把块内异常送回 yield；这里只记录后原样传播。
                # 这里捕获 BaseException 是为了也观察 CancelledError；裸 raise 保留原异常对象。
                record(events, f"所有者：yield 收到 {type(error).__name__}")
                raise
            finally:
                # 这条记录发生后才退出 aclosing，随后退出 session，轨迹可验证嵌套清理顺序。
```

```
428  async def owned_llm_stream(
459              finally:
460                  # 这条记录发生后才退出 aclosing，随后退出 session，轨迹可验证嵌套清理顺序。
461                  record(events, "所有者: 离开流的借用区间")
462
463
464  async def demonstrate_natural_consumption() -> None:
465      print_section("3. 直接 async for: 生产与消费交替, 自然结束")
466      events: list[str] = []
467      lifecycle = StreamLifecycle()
468      consumed: list[LLMEvent] = []
469
470      # 消费者只借用 stream；owned_llm_stream 才是创建并关闭 stream/session 的所有者。
471      async with owned_llm_stream("natural-session", events, lifecycle) as stream:
472          async for event in stream:
473              record(events, f"消费者: 收到 {event_label(event)}")
474              consumed.append(event)
475
476      # 能执行到这里说明管理器已完整退出；现在检查数据内容和两个资源各关闭一次。
477      print("  消费事件类型:", [type(event).__name__ for event in consumed])
478      print("  流/会话关闭次数:", lifecycle.stream_close_count, lifecycle.session_close_count)
479      assert consumed == list(FAKE_EVENTS)
480      assert lifecycle.stream_closed.is_set()
481      assert lifecycle.session_closed.is_set()
482      assert lifecycle.stream_close_count == lifecycle.session_close_count == 1
483
484      # 同一个 async for 的循环体运行完，才会请求下一项；它没有并行执行循环体。
485      # range(len(consumed) - 1) 给出每对相邻事件中前一项的下标。
486      for index in range(len(consumed) - 1):
487          consumed_marker: str = f"消费者: 收到 {event_label(consumed[index])}"
488          next_produced_marker: str = f"流: 产出 {event_label(consumed[index + 1])}"
489          assert events.index(consumed_marker) < events.index(next_produced_marker)
490
491
492  async def demonstrate_break_and_consumer_error() -> None:
493      print_section("4. break 与消费者异常: 清理责任必须写进边界")
494
```

```python
async def demonstrate_break_and_consumer_error() -> None:
    print_section("4. break 与消费者异常：清理责任必须写进边界")

    break_events: list[str] = []
    break_lifecycle = StreamLifecycle()
    # 第一部分故意不用 owned_llm_stream/aclosing，以便单独观察 async for 的 break 行为。
    async with FakeAsyncSession("break-session", break_events, break_lifecycle) as session:
        stream: AsyncGenerator[LLMEvent, None] = fake_llm_stream(session, break_events, break_lifecycle)
        async for event in stream:
            record(break_events, f"消费者：只取 {event_label(event)} 后 break")
            # break 只退出循环，不是异步 iterator 协议中的"关闭"命令。
            break

        # stream 仍被局部变量引用并暂停在 yield；其 finally 尚未执行。
        print("  break 后、aclose 前，流已关闭:", break_lifecycle.stream_closed.is_set())
        assert not break_lifecycle.stream_closed.is_set()
        # aclose 返回 awaitable；必须 await，才会向暂停的生成器注入关闭信号并跑完 finally。
        await stream.aclose()
        print("  await stream.aclose() 后，流已关闭:", break_lifecycle.stream_closed.is_set())
        assert break_lifecycle.stream_closed.is_set()

    assert break_lifecycle.session_closed.is_set()
    assert break_events.index("流：已关闭") < break_events.index("break-session：会话已关闭")

    # 第二部分改用推荐的 owning context。消费者循环体失败后，异常会穿过 yield 回到管理器；
    # aclosing 先关闭流，session 再关闭，最后外层 except 才能观察原 ConsumerError。
    error_events: list[str] = []
    error_lifecycle = StreamLifecycle()
    try:
        async with owned_llm_stream("error-session", error_events, error_lifecycle) as stream:
            async for event in stream:
                record(error_events, f"消费者：处理 {event_label(event)} 时失败")
                raise ConsumerError("渲染事件失败")
    except ConsumerError as error:
        # as error 把捕获到的异常对象绑定到局部变量，用于记录其消息。
        record(error_events, f"调用方：捕获 {error}")

```

```
492 async def demonstrate_break_and_consumer_error() -> None:
525     except ConsumerError as error:
526         # as error 把捕获到的异常对象绑定到局部变量，用于记录其消息。
527         record(error_events, f"调用方: 捕获 {error}")
528
529     assert error_lifecycle.stream_closed.is_set()
530     assert error_lifecycle.session_closed.is_set()
531     assert "所有者: yield 收到 ConsumerError" in error_events
532     assert error_events.index("流: 已关闭") < error_events.index("error-session: 会话已关闭")
533     assert error_events.index("error-session: 会话已关闭") < error_events.index(
534         "调用方: 捕获 渲染事件失败"
535     )
536
537
538 async def consume_until_cancelled(
539     events: list[str],
540     lifecycle: StreamLifecycle,
541     pause: StreamPause,
542 ) -> None:
543     # 这个函数运行在独立 Task 中。取消可能从内部某个 await 抛出 CancelledError,
544     # 但退出 owned_llm_stream 前会先走完流与会话的异步清理。
545     async with owned_llm_stream(
546         "cancel-session",
547         events,
548         lifecycle,
549         pause=pause,
550     ) as stream:
551         async for event in stream:
552             record(events, f"消费者 Task: 收到 {event_label(event)}")
553
554
555 async def demonstrate_external_cancellation() -> None:
556     print_section("5. 外部取消：先关闭流和会话，再向调用方传播")
557     events: list[str] = []
558     lifecycle = StreamLifecycle()
559     # before_index=1 表示第一项已完成消费，生成器准备第二项前停住。
560     pause = StreamPause(before_index=1)
```

```python
async def demonstrate_external_cancellation() -> None:
    print_section("5. 外部取消：先关闭流和会话，再向调用方传播")
    events: list[str] = []
    lifecycle = StreamLifecycle()
    # before_index=1 表示第一项已完成消费，生成器准备第二项前停住。
    pause = StreamPause(before_index=1)
    # create_task 立即把 coroutine 交给事件循环调度；保留 Task 引用才能取消并等待其收尾。
    consumer_task: Task[None] = asyncio.create_task(
        consume_until_cancelled(events, lifecycle, pause),
        name="cancelled-stream-consumer",
    )

    # 这道事件证明任务已经消费第一项，并在请求第二项时进入可取消等待。
    await pause.waiting.wait()
    # cancel 只是请求在 Task 的下一处可取消点抛 CancelledError，不代表此刻已经清理完成。
    consumer_task.cancel("用户停止生成")
    try:
        # 所有者必须继续 await 被取消的 Task；返回到这里时，内部 finally/__aexit__ 才已跑完。
        await consumer_task
    except asyncio.CancelledError:
        record(events, "调用方: 收到 CancelledError")
    else:
        raise AssertionError("外部取消不应被流所有者吞掉")

    print("  最终 Task.cancelled():", consumer_task.cancelled())
    print(
        "  流关闭 / 会话关闭:", lifecycle.stream_closed.is_set(), lifecycle.session_closed.is_set()
    )
    assert consumer_task.cancelled()
    assert lifecycle.stream_closed.is_set()
    assert lifecycle.session_closed.is_set()
    assert lifecycle.produced_count == 1
    # index 返回事件首次出现的位置；严格小于关系证明清理先于调用方观察取消。
    assert events.index("流: 已关闭") < events.index("cancel-session: 会话已关闭")
    assert events.index("cancel-session: 会话已关闭") < events.index("调用方: 收到 CancelledError")

```

```python

async def demonstrate_async_exit_stack() -> None:
    print_section("6. AsyncExitStack: 动态资源在后续获取失败时逆序关闭")
    events: list[str] = []
    sessions: list[FakeAsyncSession] = []

    try:
        # AsyncExitStack 适合资源数量由运行时循环决定的场景。每次成功进入一个管理器，
        # 栈就登记其 __aexit__；离开时按“后进先出”顺序逐个 await 清理。
        async with AsyncExitStack() as stack:
            for name in ("stack-A", "stack-B", "stack-fails-C"):
                # 每个会话各有独立生命周期记录；这里只有 state/exit_calls 被收集后检查。
                lifecycle = StreamLifecycle()
                session = FakeAsyncSession(
                    name,
                    events,
                    lifecycle,
                    fail_on_enter=name == "stack-fails-C",
                )
                sessions.append(session)
                # C 的 __aenter__ 抛错时，C 尚未成功登记，故栈不会调用 C.__aexit__；
                # C 由自己的 __aenter__ 回滚，栈随后按 B、A 的顺序关闭已登记资源。
                await stack.enter_async_context(session)
                session.assert_open()
    except SessionOpenError as error:
        record(events, f"调用方: 捕获动态获取失败 {error}")

    print("  三个资源最终状态:", [session.state.name for session in sessions])
    print("  三个 __aexit__ 调用次数:", [session.exit_calls for session in sessions])
    assert [session.state for session in sessions] == [
        SessionState.CLOSED,
        SessionState.CLOSED,
        SessionState.CLOSED,
    ]
    assert [session.exit_calls for session in sessions] == [1, 1, 0]
    assert events.index("stack-B: 会话已关闭") < events.index("stack-A: 会话已关闭")

```

```python
628
629 # 这对 Event 把“消费者已开始第一项”与“允许第一项结束”分开，
630 # 从而能在两者之间稳定观察上游是否偷跑生产第二项。
631 @dataclass(slots=True)
632 class SlowConsumerControl:
633     first_item_started: asyncio.Event = field(default_factory=asyncio.Event)
634     first_item_may_finish: asyncio.Event = field(default_factory=asyncio.Event)
635
636
637 async def run_slow_consumer(
638     events: list[str],
639     lifecycle: StreamLifecycle,
640     control: SlowConsumerControl,
641 ) -> None:
642     # 切片 [:3] 创建含前三个引用的新元组；事件对象本身不可变，可以安全复用。
643     source_events: tuple[LLMEvent, ...] = FAKE_EVENTS[:3]
644     async with owned_llm_stream(
645         "slow-session",
646         events,
647         lifecycle,
648         source_events=source_events,
649     ) as stream:
650         position = 0
651         async for event in stream:
652             position += 1
653             record(events, f"慢消费者: 开始处理第 {position} 项 {event_label(event)}")
654             if position == 1:
655                 # 通知控制器已经进入第一轮循环体，然后暂停本 Task。
656                 # 生成器与消费者处在同一个 Task 调用链中，因此生成器也无法恢复到下一轮。
657                 control.first_item_started.set()
658                 await control.first_item_may_finish.wait()
659             record(events, f"慢消费者: 完成第 {position} 项")
660
661
662 async def demonstrate_natural_backpressure() -> None:
663     print_section("7. 无 Queue: 慢消费者自然约束下一项生产")
664     events: list[str] = []
```

```
661
662  async def demonstrate_natural_backpressure() -> None:
663      print_section("7. 无 Queue: 慢消费者自然约束下一项生产")
664      events: list[str] = []
665      lifecycle = StreamLifecycle()
666      control = SlowConsumerControl()
667      # 单独建 Task 是为了让当前控制器能在消费者暂停时检查共享状态;
668      # 它没有把每个流事件各自并行化，内部仍只有一个顺序 async for。
669      task: Task[None] = asyncio.create_task(
670          run_slow_consumer(events, lifecycle, control),
671          name="slow-direct-consumer",
672      )
673
674      await control.first_item_started.wait()
675      # 此刻不是依靠时间猜测，而是由 Event 证明第一项循环体已开始且仍未完成。
676      print("  第一项处理未结束时，已经产出的数量:", lifecycle.produced_count)
677      assert lifecycle.produced_count == 1
678      assert f"流: 产出 {event_label(FAKE_EVENTS[1])}" not in events
679
680      # 放行后再 await Task，保证其消费完三项并退出资源上下文。
681      control.first_item_may_finish.set()
682      await task
683      print("  消费者继续后，最终产出数量:", lifecycle.produced_count)
684      assert lifecycle.produced_count == 3
685      assert lifecycle.stream_closed.is_set() and lifecycle.session_closed.is_set()
686
687
688  # QueueEnd 只属于进程内 producer/consumer 协议，用来唤醒 consumer 并让其退出。
689  # 它和上游的 DoneEvent、异步 iterator 的 StopAsyncIteration 是三个不同层次的结束信号。
690  @dataclass(frozen=True, slots=True)
691  class QueueEnd:
692      """内部管道结束哨兵；它不是 LLM 的 done 应用事件。"""
693
694
695  # 只创建一个不可变哨兵实例；producer 放入它，consumer 用 isinstance 识别它。
696  QUEUE_END = QueueEnd()
697  # Queue 中既能放业务事件，也能放内部哨兵，因此元素类型写成联合类型
```

```python

# QueueEnd 只属于进程内 producer/consumer 协议，用来唤醒 consumer 并让其退出。
# 它和上游的 DoneEvent、异步 iterator 的 StopAsyncIteration 是三个不同层次的结束信号。
@dataclass(frozen=True, slots=True)
class QueueEnd:
    """内部管道结束哨兵；它不是 LLM 的 done 应用事件。"""


# 只创建一个不可变哨兵实例；producer 放入它，consumer 用 isinstance 识别它。
QUEUE_END = QueueEnd()
# Queue 中既能放业务事件，也能放内部哨兵，因此元素类型写成联合类型。
type QueueItem = LLMEvent | QueueEnd


# 该状态对象是两个子 Task 与管道所有者之间的协调面：Event 表示阶段事实，
# 普通字段保存最大容量和消费结果。它只在一个事件循环线程内协作使用，不是线程同步对象。
@dataclass(slots=True)
class QueueRunState:
    producer_backpressured: asyncio.Event = field(default_factory=asyncio.Event)
    start_consuming: asyncio.Event = field(default_factory=asyncio.Event)
    pipeline_ready: asyncio.Event = field(default_factory=asyncio.Event)
    producer_finished: asyncio.Event = field(default_factory=asyncio.Event)
    consumer_finished: asyncio.Event = field(default_factory=asyncio.Event)
    max_queue_size: int = 0
    # list 是可变对象，必须用 default_factory；否则多个 QueueRunState 会共享消费结果。
    consumed: list[LLMEvent] = field(default_factory=list)


async def queue_producer(
    queue: asyncio.Queue[QueueItem],
    events: list[str],
    lifecycle: StreamLifecycle,
    state: QueueRunState,
) -> None:
    """producer 拥有底层流；取消或失败时上下文仍会关闭流和会话。"""
    try:
        # producer 是底层 stream 的唯一消费者与所有者：它把应用事件转交到 Queue，
```

```python
async def queue_producer(
    queue: asyncio.Queue[QueueItem],
    events: list[str],
    lifecycle: StreamLifecycle,
    state: QueueRunState,
) -> None:
    """producer 拥有底层流；取消或失败时上下文仍会关闭流和会话。"""
    try:
        # producer 是底层 stream 的唯一消费者与所有者；它把应用事件转交到 Queue。
        async with owned_llm_stream("queue-session", events, lifecycle) as stream:
            async for event in stream:
                # full() 只是当前瞬间的观察。此实验只有一个 producer，且 consumer 尚未放行，
                # 所以 set() 可稳定证明接下来的 put 会因容量已满而等待。
                if queue.full():
                    record(events, f"Queue producer: 队列已满({queue.qsize()}), 等待 put")
                    state.producer_backpressured.set()
                # 有界 Queue 已满时 await put 会暂停 producer，直到 consumer 腾出一格。
                await queue.put(event)
                # qsize 是当前缓冲数量；max 保存整个实验实际观察到的容量峰值。
                state.max_queue_size = max(state.max_queue_size, queue.qsize())
                record(
                    events, f"Queue producer: 已放入 {event_label(event)}, qsize={queue.qsize()}"
                )

        # 只有流自然结束，才发送内部哨兵。若 producer 异常或取消，TaskGroup 会取消 consumer，
        # 不能假定任何失败路径都一定来得及把哨兵放进可能已满的 Queue。
        await queue.put(QUEUE_END)
        state.max_queue_size = max(state.max_queue_size, queue.qsize())
        record(events, "Queue producer: 已放入内部结束哨兵")
    except asyncio.CancelledError:
        # 只记录后重新抛出，保留 Task 的取消状态，让 TaskGroup/调用方正确观察失败。
        record(events, "Queue producer: 收到取消")
        raise
    finally:
        # 无论正常、异常或取消，都发布“producer 已结束”的可观察事实。
        state.producer_finished.set()
```

```
715  async def queue_producer(
748      finally:
749          # 无论正常、异常或取消，都发布“producer 已结束”的可观察事实。
750          state.producer_finished.set()
751          record(events, "Queue producer: 已结束")
752
753
754  async def queue_consumer(
755      queue: asyncio.Queue[QueueItem],
756      events: list[str],
757      state: QueueRunState,
758  ) -> None:
759      """每次 get 都与一次 task_done 配对，包括内部结束哨兵。"""
760      try:
761          # 正常实验先让 producer 填满队列；取消实验则始终不打开这道闸门。
762          await state.start_consuming.wait()
763          while True:
764              # Queue 为空时 get 会等待。尚未完成数在 put 时增加；get 只取走项目，
765              # 要到下面的 task_done 才会把对应记账减一。
766              item = await queue.get()
767              try:
768                  if isinstance(item, QueueEnd):
769                      record(events, "Queue consumer: 收到内部结束哨兵")
770                      # return 仍会先执行内层 finally，所以哨兵也能正确 task_done。
771                      return
772                  # 经过 isinstance 排除 QueueEnd 后，类型检查器也能把 item 缩小为 LLMEvent。
773                  state.consumed.append(item)
774                  record(events, f"Queue consumer: 消费 {event_label(item)}, qsize={queue.qsize()}")
775              finally:
776                  # task_done 只表示这一次 get 的 Queue 记账结束，不等于业务一定成功。
777                  queue.task_done()
778
779              # 给 producer 一个推进机会；正确性断言仍只依赖 Event 与队列状态。
780              await asyncio.sleep(0)
781      except asyncio.CancelledError:
782          record(events, "Queue consumer: 收到取消")
```

```
754 async def queue_consumer(
781     except asyncio.CancelledError:
782         record(events, "Queue consumer: 收到取消")
783         raise
784     finally:
785         state.consumer_finished.set()
786         record(events, "Queue consumer: 已结束")
787
788
789 async def demonstrate_bounded_queue() -> None:
790     print_section("8. 有界 Queue: 解耦生产消费, 同时保留容量背压")
791     events: list[str] = []
792     lifecycle = StreamLifecycle()
793     state = QueueRunState()
794     # maxsize=2 表示最多缓冲两项；默认值 0 反而表示无界，不是零容量。
795     queue: asyncio.Queue[QueueItem] = asyncio.Queue(maxsize=2)
796
797     # TaskGroup 共同拥有 producer 与 consumer：退出上下文会等待二者；任一子任务以普通异常
798     # 失败时，会取消同组其他任务并在收尾后向外传播异常组。
799     async with asyncio.TaskGroup() as group:
800         producer_task: Task[None] = group.create_task(
801             queue_producer(queue, events, lifecycle, state),
802             name="bounded-queue-producer",
803         )
804         consumer_task: Task[None] = group.create_task(
805             queue_consumer(queue, events, state),
806             name="bounded-queue-consumer",
807         )
808
809         # consumer 仍在闸门外；producer 填满两格后会确定地阻塞在第三次 put。
810         await state.producer_backpressured.wait()
811         print("  producer 等待时 qsize/maxsize:", queue.qsize(), queue.maxsize)
812         assert queue.qsize() == queue.maxsize == 2
813         assert not producer_task.done()
814         assert not consumer_task.done()
815         # set 后 consumer 的 wait 可通过，开始 get；腾出的容量让 producer 的 put 继续。
816         state.start_consuming.set()
```

```
789 async def demonstrate_bounded_queue() -> None:
814         assert not consumer_task.done()
815         # set 后 consumer 的 wait 可通过，开始 get；腾出的容量让 producer 的 put 继续。
816         state.start_consuming.set()
817
818     # 越过 TaskGroup 时两个 Task 都已结束。join 再核对每次 put 都已有匹配的 task_done。
819     await queue.join()
820     print("  观察到的最大队列长度:", state.max_queue_size)
821     print("  producer / consumer 已结束:", producer_task.done(), consumer_task.done())
822     assert state.consumed == list(FAKE_EVENTS)
823     assert state.max_queue_size == 2
824     assert state.producer_finished.is_set() and state.consumer_finished.is_set()
825     assert lifecycle.stream_closed.is_set() and lifecycle.session_closed.is_set()
826
827
828 async def cancellable_queue_pipeline(
829     events: list[str],
830     lifecycle: StreamLifecycle,
831     state: QueueRunState,
832 ) -> None:
833     """管道所有者在取消后回收双方，并明确丢弃尚未消费的队列项。"""
834     queue: asyncio.Queue[QueueItem] = asyncio.Queue(maxsize=2)
835     try:
836         # 父管道拥有整个 TaskGroup。外部取消父 Task 时，取消会进入这个上下文，
837         # TaskGroup 再取消并等待 producer/consumer，之后才允许执行外层 finally。
838         async with asyncio.TaskGroup() as group:
839             group.create_task(
840                 queue_producer(queue, events, lifecycle, state),
841                 name="cancel-queue-producer",
842             )
843             group.create_task(
844                 queue_consumer(queue, events, state),
845                 name="cancel-queue-consumer",
846             )
847             # 等 producer 准备向满队列继续 put，确保两个子任务都停在明确的可取消点。
848             await state.producer_backpressured.wait()
```

```
async def cancellable_queue_pipeline(
            # 等 producer 准备向满队列继续 put，确保两个子任务都停在明确的可取消点。
            await state.producer_backpressured.wait()
            state.pipeline_ready.set()
            # 新建的 Event 永不 set，因此这里会一直等待，直到外部对父 Task 发出取消。
            await asyncio.Event().wait()
    finally:
        # 本例的取消策略是“丢弃尚未交付项”。每个已 put 项仍要完成 task_done
        # 记账；否则 queue.join() 会永久等待。本地队列随作用域一起销毁。
        discarded = 0
        while not queue.empty():
            # get_nowait 在已确认非空后同步取出一项；这里没有其他仍运行的 Queue 使用者竞争。
            queue.get_nowait()
            queue.task_done()
            discarded += 1
        # 所有残留项都补完记账后 join 应立即返回，也把“没有遗漏 task_done”写成可执行检查。
        await queue.join()
        record(events, f"Queue owner: 丢弃 {discarded} 项并清空 join 记账")


async def demonstrate_queue_cancellation() -> None:
    print_section("9. Queue 管道取消：producer、consumer、流和会话一起回收")
    events: list[str] = []
    lifecycle = StreamLifecycle()
    state = QueueRunState()
    # 与单个消费者取消实验相同：必须保留父 Task，cancel 后还要 await 它完成级联清理。
    pipeline_task: Task[None] = asyncio.create_task(
        cancellable_queue_pipeline(events, lifecycle, state),
        name="cancellable-queue-pipeline",
    )

    await state.pipeline_ready.wait()
    # pipeline_ready 证明队列已满、producer 卡在 put、consumer 卡在启动闸门。
    pipeline_task.cancel("请求已断开")
    try:
        await pipeline_task
```

```
866  async def demonstrate_queue_cancellation() -> None:
880      try:
881          await pipeline_task
882      except asyncio.CancelledError:
883          record(events, "Queue 调用方: 收到 CancelledError")
884      else:
885          raise AssertionError("管道所有者不应吞掉外部取消")
886
887      print(
888          "  producer / consumer finished:",
889          state.producer_finished.is_set(),
890          state.consumer_finished.is_set(),
891      )
892      print(
893          "  stream / session closed:",
894          lifecycle.stream_closed.is_set(),
895          lifecycle.session_closed.is_set(),
896      )
897      assert pipeline_task.cancelled()
898      assert state.producer_finished.is_set()
899      assert state.consumer_finished.is_set()
900      assert lifecycle.stream_closed.is_set()
901      assert lifecycle.session_closed.is_set()
902      # next(generator expression) 找到具体的 owner 清理记录；若没有该记录会直接失败。
903      owner_cleanup: str = next(event for event in events if event.startswith("Queue owner: 丢弃"))
904      assert events.index("Queue producer: 已结束") < events.index(owner_cleanup)
905      assert events.index("Queue consumer: 已结束") < events.index(owner_cleanup)
906      assert events.index(owner_cleanup) < events.index("Queue 调用方: 收到 CancelledError")
907
908
909  async def run_examples() -> None:
910      # 前两段分别是异步与同步演示，所以只有 coroutine 调用需要 await。
911      # 后续段落按“正常 -> 提前退出/异常 -> 取消 -> 动态资源 -> 背压 -> Queue”递进执行。
912      await demonstrate_protocol_and_object_kinds()
913      demonstrate_transport_boundaries()
914      await demonstrate_natural_consumption()
```

```python

async def run_examples() -> None:
    # 前两段分别是异步与同步演示，所以只有 coroutine 调用需要 await。
    # 后续段落按"正常 -> 提前退出/异常 -> 取消 -> 动态资源 -> 背压 -> Queue"递进执行。
    await demonstrate_protocol_and_object_kinds()
    demonstrate_transport_boundaries()
    await demonstrate_natural_consumption()
    await demonstrate_break_and_consumer_error()
    await demonstrate_external_cancellation()
    await demonstrate_async_exit_stack()
    await demonstrate_natural_backpressure()
    await demonstrate_bounded_queue()
    await demonstrate_queue_cancellation()

    # asyncio.all_tasks() 返回当前事件循环中尚未结束的 Task 集合，其中包含当前 run_examples。
    # 集合推导式排除当前 Task 和已完成 Task；空集合证明示例没有遗留后台工作。
    unfinished: set[Task[Any]] = {
        task
        for task in asyncio.all_tasks()
        if task is not asyncio.current_task() and not task.done()
    }
    print("\n入口结束前未回收的其他 Task 数:", len(unfinished))
    assert not unfinished
    print("所有第 24 章示例与运行时断言均已完成。")


def main() -> None:
    # asyncio.run 创建事件循环、运行顶层 coroutine，并在返回前完成 runner 级别的收尾。
    # 资源仍由各实验在自己的局部 async-with 中关闭，不能依赖入口退出才回收。
    asyncio.run(run_examples())


# 导入该模块时只定义类型与函数；直接执行本文件时，特殊变量 __name__ 才等于 "__main__"。
if __name__ == "__main__":
    main()

```